

Export, Deployment i Model API

Jun 2023.

1 Eksportovanje modela

Eksportovanje modela predstavlja proces čuvanja već istreniranog modela u fajl, tako da se može koristiti van okruženja u kojem je treniran. Ova faza je ključna za omogućavanje **produkcione primene**.

Bez eksportovanja, model ostaje "zarobljen" u radnom okruženju i ne može se koristiti u stvarnim aplikacijama. Eksportovanje omogućava:

- Primenu modela na novim, neviđenim podacima u produkciji
- Deljenje modela sa drugim istraživačima ili timovima
- Uštedu vremena eliminisanjem potrebe za ponovnim treniranjem

1.1 Kako se model eksportuje?

Proces eksportovanja zavisi od korišćenog okvira i željenog formata, ali obično uključuje sledeće korake:

1. Treniranje i *fine-tuning* modela
2. Izbor formata za eksport
3. Cuvanje modela u odabranom formatu
4. Testiranje eksportovanog modela radi validacije

1.2 Formati za eksportovanje modela

Postoji više različitih formata za eksportovanje modela, od kojih svaki ima svoje prednosti i ograničenja:

- **Pickle (.pkl)** - Python-ov binarni format za serijalizaciju objekata. Lako se koristi, ali nije prenosiv van Python okruženja i može biti ranjiv ako se koristi sa nepouzdanim fajlovima.

- **ONNX (Open Neural Network Exchange)** - Otvoreni format za interoperabilnost modela između različitih biblioteka i jezika. Pogodan za rad u okruženjima kao što su C++, Java i JavaScript.
- **TensorFlow SavedModel** - Format specifičan za TensorFlow koji omogućava snimanje kompletne strukture modela, težina i konfiguracije. Pogodan za TensorFlow Serving.
- **PMML (Predictive Model Markup Language)** - XML-baziran format koji omogućava razmenu modela između različitih jezika i platformi (npr. Java, R, C++). Koristan za modele kao što su stabla odlučivanja, logistička regresija i SVM.
- **PyTorch (.pth/.pt)** - Format koji omogućava čuvanje samo težina (state_dict) ili kompletnog PyTorch modela. Takođe podržava eksport u ONNX format.

1.3 Izbor formata

Izbor formata za eksport zavisi od više faktora:

- **Korišćeni framework** - Na primer, TensorFlow koristi SavedModel, dok PyTorch koristi .pth.
- **Ciljno okruženje** - Ako se model koristi u aplikaciji pisanom u drugom jeziku (npr. C++), ONNX ili PMML su bolji izbor.
- **Veličina modela i performanse** - Neki formati su kompaktniji i brže se učitavaju (npr. ONNX).
- **Bezbednosni zahtevi** - Pickle može biti rizičan za deljenje preko mreže ako fajlovi nisu verifikovani.
- **Mesto izvršavanja modela:**
 - **Edge uređaji:** Za uređaje sa ograničenim resursima (poput senzora, mobilnih telefona), potrebno je koristiti lagane i efikasne formate. Često se koristi kvantizacija i formati poput ONNX ili TensorFlow Lite.
 - **Cloud:** Kada se model izvršava u cloud-u, može se koristiti bilo koji format koji je podržan od strane cloud servisa (npr. SavedModel za TensorFlow Serving na GCP ili AWS).
 - **Lokalni serveri (on-premise):** U ovom slučaju je poželjno koristiti format koji je lako integrisiv u postojeću infrastrukturu organizacije, kao što su PMML, ONNX ili čak Pickle ako se koristi Python backend.

2 Docker i kontejnerizacija modela

2.1 Docker

Docker je otvorena platforma za razvoj, isporuku i izvršavanje aplikacija. Omogućava programerima da svoje aplikacije izoluju od infrastrukture na kojoj se izvršavaju, čime se ubrzava i standardizuje proces isporuke softverskih rešenja.

Docker koristi **kontejnere**, koji predstavljaju lagana, izolovana okruženja za izvršavanje aplikacija. Kontejner sadrži sve što je aplikaciji potrebno (kod, biblioteke, konfiguracije), tako da ne zavisi od podešavanja računara na kojem se izvršava. Pruža sledeće prednosti:

- **Brza i dosledna isporuka aplikacija** - Kontejneri omogućavaju da aplikacija uvek radi na isti način, bez obzira na okruženje.
- **Izolacija** - Više kontejnera može da se izvršava paralelno na istom računaru, bez međusobnog ometanja.
- **Prenosivost** - Jednom kreiran kontejner može da se koristi bilo gde: na lokalnom računaru, na cloud-u ili na lokalnim serverima (on-premise).
- **Podrška za CI/CD** - Docker se savršeno uklapa u tokove kontinuirane integracije i isporuke, jer omogućava automatizaciju testiranja i deployovanja.

2.2 Ključne komponente Dockera

Da bi se Docker uspešno koristio, važno je razumeti osnovne komponente koje čine njegov ekosistem:

- **Docker Engine** - Glavni deo Docker platforme koji omogućava kreiranje i upravljanje kontejnerima.
- **Docker Image** - Read-only šablon koji sadrži sav potreban kod, biblioteke i zavisnosti. Na osnovu slike (image-a) se kreira kontejner (kontejner je runtime instanca image-a).
- **Dockerfile** - Tekstualni fajl koji definiše korake potrebne da se napravi Docker image (npr. instalacija Python paketa, kopiranje modela, itd).
- **Docker Hub** - Cloud repozitorijum za deljenje i preuzimanje gotovih Docker image-a. Sličan Githubu-u, ali za kontejnere.

2.3 Primena u mašinskom učenju

Docker se sve češće koristi i za **deployovanje modela mašinskog učenja**. Nakon treniranja modela, on se zajedno sa API-jem (npr. Flask ili FastAPI) pakuje u Docker kontejner i može se preneti na željeno okruženje. Na ovaj način, model postaje lako distribuiran i nezavisan od konkretne infrastrukture.

3 Model API

API (Application Programming Interface) predstavlja interfejs koji omogućava komunikaciju između različitih softverskih komponenti. U kontekstu mašinskog učenja, **Model API** je servis koji omogućava da se model koristi van okruženja u kojem je treniran. Putem API-ja, drugi sistemi mogu da šalju podatke modelu i dobijaju njegove predikcije bez direktnog uvida u način na koji model funkcioniše. API je ključna komponenta u procesu implementacije modela, jer omogućava:

- **Integraciju sa drugim sistemima** - Web aplikacije, mobilne aplikacije ili baze podataka mogu da šalju zahteve modelu i dobiju rezultat.
- **Real-time predikcije** - Model može odmah da odgovori na nove podatke u realnom vremenu.
- **Jednostavno deployovanje** - Model može lako da se postavi na server ili cloud platformu kao servis.
- **Izolaciju modela** - Korisnici ne moraju da znaju detalje implementacije, dovoljno je da znaju ulaz i izlaz.
- **Skalabilnost i sigurnost** - API omogućava da se pristup modelu kontroliše, ograniči i prati.

U savremenim aplikacijama, model bez API-ja ostaje ograničen na lokalnu upotrebu i ne može biti iskorišćen u stvarnim aplikacijama. API omogućava modelu da postane servis koji se lako koristi u drugim sistemima.

Model API je najčešće dizajniran tako da obezbedi osnovne funkcionalnosti neophodne za upotrebu i praćenje modela u produkcionom okruženju. Tipične komponente uključuju:

- **Predikciju** - Prima ulazne podatke i vraća rezultat modela.
- **Praćenje istorije predikcija** - Čuva se log svakog zahteva i odgovora, korisno za analizu, debugging, izvestaje itd.
- **Informacije o modelu** - Verzija modela, tačnost ili neke druge statistike.
- **Monitoring i zdravlje servisa** - Proverava da li API i model funkcionišu ispravno.